

COL380 Assignment 3 : Budgeted Max Weight Clique using MPI

Degala Aman (2023CS50838)

12 April 2026

1 Performance Analysis

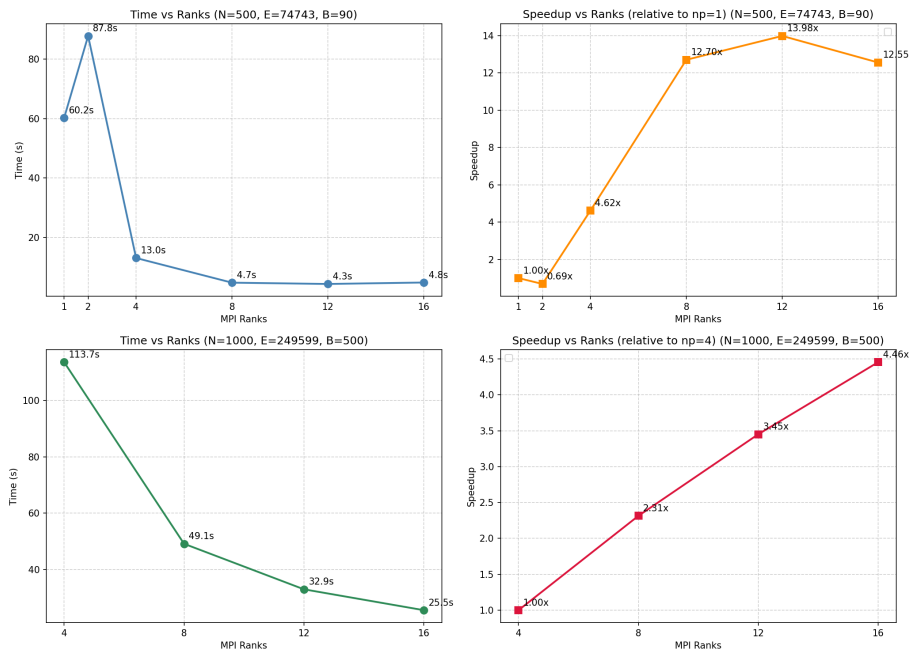


Figure 1: Execution time and speedup vs number of MPI ranks for two different inputs

for the first input(N=500 , E=74743 , B=90) np=2 is slower than np=1 because in np=2 , it has the same number of workers as np=1 but now there is overhead of message sending and recieving of mpi.

at about np=12, further increase in number of processes doesnt improve the speed for this particular input

for the second input , time for np=1,2 is too high and the speedup doesnt plateau at n=16 like it did for the previous input since N and E have high values.

2 Parallelization Strategy

When we run the MPI program with `np > 1`, the process with `rank = 0` acts as a master and the rest are workers. The master first builds all the tasks and keeps allotting them to workers as and when the workers finish their currently assigned task.

We use four different message tags to communicate between the master and workers:

1. `TAG_TASK` : The master sends a task to a worker..
2. `TAG_NO_MORE_WORK` : When all the tasks are allotted to the workers and a worker complete a task, the master sends this to the worker.
3. `TAG_RESULT` : Worker sends its local best clique and profit using this tag.
4. `TAG_PMAX_UPDATE` : Whenever a worker finds a profit greater than the global profit it uses this tag to send it to the master and then the master sends it to the rest to update their global maximums and prune better.

The initial tasks are generated by allotting the branches of the first branching to each process. For each vertex v , we create task with v in the clique and candidate set is all the adjacent vertices to v .

The workers do the further computation on the given local set and find the maximum profit clique within the budget. In between of this computation if it finds that its profit exceeds that of the max global profit, it uses the `TAG_PMAX_UPDATE` to send this new max global profit to the master.

We use buffering to send messages between both master and worker to avoid cases of deadlock where both master and worker are stuck in sending the message.